MDQ Specification Audit: True/False: mdq-spec

Date: 2026-10-16

Scope: The true-false question type: `docs/question-types/true-false.md`, `schema/true-false.yaml`, the shared choice list rules in `docs/references/choices.md`, and the parts of `docs/references/grammar.md`, `docs/question-types/question-base.md`, `docs/exam.md`, `docs/responses.md` and the corpus that concern true-false.

Methodology

Read docs/references/grammar.md, docs/references/patterns.md, docs/question-types/question-base.md, docs/exam.md, docs/responses.md, docs/references/choices.md and docs/question-types/true-false.md, and checked suspicious rules against the corpus (examples/valid/true-false/, examples/invalid/true-false-*, examples/grading/true-false.yaml) and the reference implementation (mdq-py, `mdq validate`).

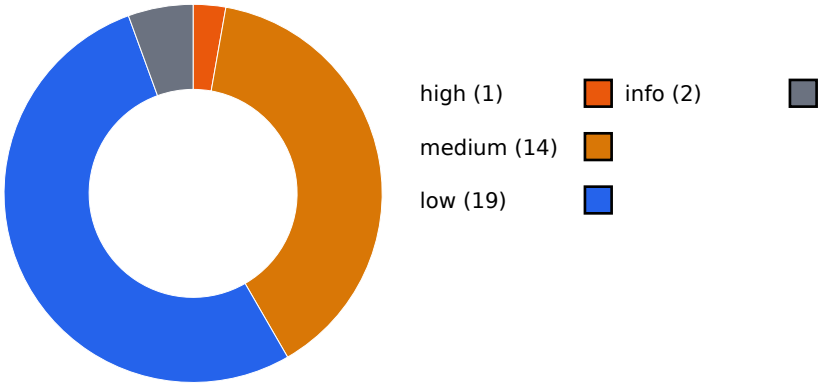
A grammar subagent rebuilt the true/false item grammar in Lark, with the TRUE, FALSE and WARNING letter sets built from Unicode General_Category, and tested what it accepts against the prose: letters, case, CJK, symbols, spaces, Unicode spaces, links after the marker, choice ids, feedback and comment blocks, and type inference on mixed lists.

A second subagent scanned true-false.md independently: prose against the Additional Rules table (MUST/SHOULD/MAY against error/warning/info), against schema/true-false.yaml, docs/lint-codes.md and the corpus, and validated the complete example with mdq-py. scripts/spec_audit.py passed before and after the audit.

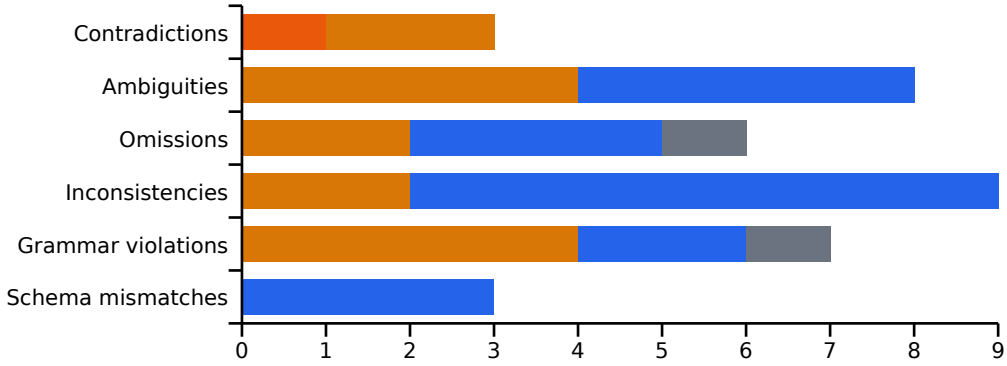
Executive summary

Severity	#
CRITICAL	0
HIGH	1
MEDIUM	14
LOW	19
INFO	2
Total findings	**36**

Findings by severity



Findings by category



Strengths and weaknesses

Strengths

✓ **Omissions:** The `marker`/`correct` split is well argued: `correct` drives grading, `marker` keeps round-tripping and enables the WARNING and locale lints, and the spec says which lints are lost without `marker`. (docs/question-types/true-false.md:107-144)

✓ **Contradictions:** The symmetric and partial rows of the grading table agree with the prose and with examples/grading/true-false.yaml, including abstentions and explicit-null cases. (docs/question-types/true-false.md:213-225)

✓ **Omissions:** The corpus covers every true/false lint code in the Additional Rules table, including CJK, non-BMP, non-NFC, false-friend lowercase and locale without expectation (examples/valid/true-false/). (docs/question-types/true-false.md:228-252)

✓ **Inconsistencies:** Feedback rules for true/false (wrong and abstained statements, document order, none on skip) agree with GLOSSARY.md Feedback and docs/responses.md Skipping. (docs/question-types/true-false.md:177-189)

✓ **Grammar violations:** The true-false `value` rule, with `true : /[TtVvSsU+5BF9U+771F]/`, `false : /[FfU+9519U+507D]/` and the WARNING class built from General_Category, accepts `[T]`, `[t]`, `[V]`, `[S]`, `[F]`, `[f]`, `[U+5BF9]`, `[U+771F]`, `[U+9519]`, `[U+507D]`, `[ T ]`, `[\tT\t]`, `[é]` (U+00E9) and the astral letter U+20000, and rejects `[X]`, `[x]`, `[TF]`, `[ ]`, `[]`, `[✓]`, `[×]`, `[U+3007]`, `[1]`, `[*]` and `[e\u0301]`, as the prose states (Lark Earley test). (docs/question-types/true-false.md:43-98)

✓ **Grammar violations:** `ws` is only space and tab, so `[\u00a0T]` and `[\u3000F]` fail the `value` rule, which agrees with the rule that a Unicode space where the grammar expects `ws` is text. (docs/references/grammar.md:28-42)

✓ **Grammar violations:** `VALUE_END` refuses a bracket followed by `(` or `[:` `* [T](http://x) a` and `* [T][a] a` fail the `item` rule and stay links, in line with `CHOICE_MARK` and `CHOICE_ID`. (docs/references/choices.md:15-23, docs/references/choices.md:56-57)

✓ **Grammar violations:** The `item` grammar is ambiguous about the `ws` after the value and about whether `[brasilia]` is a `CHOICE_ID` or text (2 and 4 parse trees in the Earley test), but the prose settles both: the bracket that matches `CHOICE_ID` is always the id, and the text excludes the `ws` after the value and the id. `* [T] [brasilia]` gives an empty text and `* [T] [V] text` gives the id `V`. (docs/references/choices.md:68-70, docs/references/choices.md:109-111)

✓ **Grammar violations:** Feedback and comment rules hold for true-false items in the test: `> fb` then `! c` is accepted, a lazy line continues the feedback, a blank line before the feedback is accepted, an empty `>` line is part of the block, `![x](y.png)` is a lazy line and not a comment, `2. two` is a lazy line and `1. one` is not, and a second `>` block after a comment fails the rule (the stated `parse-error`). (docs/references/grammar.md:116-161)

✓ **Grammar violations:** Type inference decides mixed lists: `[x]` with `[T]` is multiple selection by rule 2 and `[T]` is a `foreign-choice-marker`; `[ ]` with `[F]` is true/false by rule 3 and `[ ]` is a `foreign-choice-marker` (as `examples/invalid/true-false-blank-marker` expects); `[✓]`/`[✗]` falls to rule 4. The X/x exclusion of the WARNING set matches rule 2, so no letter is claimed by both types. (docs/question-types/question-base.md:229-245, docs/question-types/true-false.md:85)

✓ **Grammar violations:** No compatibility ideograph whose NFC form is a TRUE or FALSE letter changes the meaning: the only ones found, U+2F946 and U+2F947 (NFC U+771F), are WARNING letters and so also mean true (scan of all `\p{L}` code points with `unicodedata`, Unicode 15.1). (docs/question-types/true-false.md:83-86)

✓ **Schema mismatches:** The true/false schema agrees with the spec on `minItems: 2`, the slug pattern of `id`, the `grading` enum, `shuffle` as boolean or `inherit`, `correct` required and `additionalProperties: false` on statements. (schema/true-false.yaml:17-111)

✓ **Contradictions:** The only complete true/false document in the spec (the Example section) validates with mdq-py with no warning. (docs/question-types/true-false.md:9-16)

Central risks

• The letter test of true/false depends on the Unicode database and the case mapping of each runtime, so Python and JavaScript can disagree on the type and the answer key of the same document.

• The same strategy name (`all-or-nothing`) means different things in true/false and in the other types, so an exam-wide `grading` gives inconsistent results.

Detailed findings

Contradictions

Severity	Location	Description
HIGH	docs/question-types/true-false.md:204-207	**F1** True/false all-or-nothing gives partial credit
MEDIUM	docs/question-types/true-false.md:173-175	**F2** Locale text refers to an `id` row that the table does not have
MEDIUM	docs/lint-codes.md:72, docs/question-types/question-base.md:103-107	**F32** `docs/lint-codes.md` makes a `null` feedback or comment a `schema-error`

F1: True/false all-or-nothing gives partial credit

Location: docs/question-types/true-false.md:204-207

```
* **all-or-nothing**: The student must mark all choices with the correct marking
  or abstain. Any mistake in the marked choices gives zero points. If all marked
  choices are correct, the student gets a score proportional to the number of
  correct markings (`correctly marked / total number of choices`).
```

Description: Under true/false `all-or-nothing`, a response with no mistakes scores `correctly marked / total`, so `[T, F, _, _]` scores 0.5 (table at line 223, and `examples/grading/true-false.yaml:45-46`). GLOSSARY.md defines All-or-nothing as 'scoring 1 only when every part is correct, and 0 otherwise', and multiple selection (`docs/question-types/multiple-selection.md:114-115`) and fill-in (`docs/question-types/fill-in.md:327`) follow the glossary. The true/false rule is 'partial, unless there is a mistake'.

Impact: An exam that sets `grading: all-or-nothing` for every type gets two different meanings. An instructor who expects no credit for an incomplete true/false answer gives 0.5 to a student who judged half of the statements.

Suggested fix: Either make true/false all-or-nothing score 1 only when every statement is judged correctly (an abstention counts as a mistake), or keep the current rule, give it a different name, and update the glossary. The first option agrees with the glossary and the other types.

F2: Locale text refers to an `id` row that the table does not have

Location: docs/question-types/true-false.md:173-175

```
dedicated warning, `false-friend-true-false-marker`. The `id` row has no
mismatched form, so `locale-mismatched-true-false-marker` is never issued for
it.
```

Description: The locale table at lines 160-164 has rows only for en, pt and es. There is no `id` row. Line 155-157 says a language not in the table has no expectation, so `id` already gets no `locale-mismatched-true-false-marker`; the sentence about an `id` row with 'no mismatched form' describes a row that does not exist.

Impact: A reader cannot tell whether `id` is meant to be in the table (with which letters) or not.

Suggested fix: Delete the sentence about the `id` row, or add an `id` row (B / S? T / F?) and define what 'no mismatched form' means.

F32: `docs/lint-codes.md` makes a `null` feedback or comment a `schema-error`

Location: docs/lint-codes.md:72, docs/question-types/question-base.md:103-107

Mechanical fix

```
`null` in these fields is a `schema-error`.
```

Description: The `blank-text` row lists the `feedback` and `comment` of a choice, which are OPTIONAL, and ends with '`null` in these fields is a `schema-error`'. question-base.md:103-107 says a `null` in any OPTIONAL field, including nested objects such as choices, is the same as an absent field, and question-base.md:349 repeats it. mdq-py accepts a true/false statement with `feedback: null`.

Impact: An implementation that follows `docs/lint-codes.md` rejects `feedback: null`, which the base spec and the reference implementation accept.

Suggested fix: Change the sentence to: '`null` in an OPTIONAL field is an absent field; in a required field it is a `schema-error`.'

Exploitability conditions: A YAML or JSON document with `feedback: null` or `comment: null` on a choice.

Ambiguities

Severity	Location	Description
MEDIUM	docs/question-types/true-false.md:149-153	**F3** The spec does not define which markers are locale-mismatched
MEDIUM	docs/question-types/true-false.md:65-68, docs/question-types/true-false.md:251	**F5** NFC rule mixes combining sequences with non-NFC single code points
MEDIUM	docs/question-types/true-false.md:46-50, docs/question-types/true-false.md:83-86, docs/question-types/true-false.md:96-98	**F6** Case-insensitivity and compatibility forms of letters are not defined
MEDIUM	docs/question-types/true-false.md:113, docs/question-types/true-false.md:137-139	**F27** `marker` is 'exactly as it was written', but the ws inside the brackets is dropped
LOW	docs/question-types/true-false.md:256-257	**F10** False-friend footnote speaks of an S 'meaning false'
LOW	docs/question-types/question-base.md:230-234	**F12** Type inference 'a value is a letter' does not say spaces are removed

Severity	Location	Description
LOW	docs/question-types/true-false.md:5-7	**F35** The introduction calls true statements 'correct answers'
LOW	docs/question-types/true-false.md:87-88	**F37** S is called the initial of words written in non-Latin scripts

F3: The spec does not define which markers are locale-mismatched

Location: docs/question-types/true-false.md:149-153

```
Implementations MAY issue warnings for True/False letters that are incompatible
with the locale for the question. For instance, if `locale: pt-BR`, it may
expect `V` and `F` -- for "verdadeiro" (true) and "falso" (false) -- and could
issue a warning if `T` is used instead.
```

Description: 'Incompatible with the locale' is defined only by one example (T under `pt`). Open cases: is S, U+5BF9 or U+9519 under `pt` mismatched? Is a WARNING letter such as `W` under `pt` also mismatched, or only `unlisted-true-false-marker`? Is lowercase `t` mismatched (case-insensitive)? mdq-py warns for every TRUE or FALSE letter outside the expected pair, case-insensitively, and never for a WARNING letter: under `pt-BR` it flags S, `t`, U+5BF9, U+9519 and not `W`.

Impact: Two implementations can report different warnings for the same document.

Suggested fix: State the rule: under a locale in the table, a marker in the TRUE or FALSE category whose case-folded form is not the expected letter of its category is mismatched. WARNING letters get only `unlisted-true-false-marker`.

Exploitability conditions: `locale` with a language subtag in the table.

F5: NFC rule mixes combining sequences with non-NFC single code points

Location: docs/question-types/true-false.md:65-68, docs/question-types/true-false.md:251

```
* "One code point", not one grapheme cluster. A base letter followed by a
  combining mark is two code points and is not a valid marker. Implementations
  MAY report, at the info level, a marker that is not in NFC form: in NFC,
  precomposed forms are single code points.
```

Description: The bullet says a combining sequence is invalid (an error), then in the same breath offers an info for a marker 'not in NFC form', with the reason 'in NFC, precomposed forms are single code points'. That reason is about combining sequences, which are already `malformed-true-false-marker`. The info can only apply to a single code point whose NFC form differs, such as U+212B ANGSTROM SIGN (NFC U+00C5) or U+212A KELVIN SIGN (NFC K), which is the case `examples/valid/true-false/non-nfc-marker.yaml` tests and `docs/lint-codes.md:39` describes. The table row 'is not a single code point in NFC form' can be read as covering combining sequences too.

Impact: An implementer can emit the info for combining sequences, or miss the singleton case.

Suggested fix: Split the bullet: a combining sequence is two code points and is `malformed-true-false-marker`; a single code point that NFC changes (U+212B to U+00C5) MAY get `non-nfc-true-false-marker`. Reword the table row to 'is a single code point that NFC changes'.

F6: Case-insensitivity and compatibility forms of letters are not defined

Location: docs/question-types/true-false.md:46-50, docs/question-types/true-false.md:83-86, docs/question-types/true-false.md:96-98

```
We only show the upper case versions of each symbol. Each set must be expanded
with lowercase versions as well; the CJK characters have no case and are used
as-is.
```

Description: The spec does not say which case mapping applies. mdq-py uses case folding: `[ſ]` (U+017F LATIN SMALL LETTER LONG S) folds to s and reads as TRUE with no warning. The fullwidth letters that CJK input methods produce are WARNING letters: `[U+FF26]` (U+FF26) reads as true with only `unlisted-true-false-marker`, so a statement the author marked false is graded as true. A titlecase letter (`Lt`, such as U+01C5) is a WARNING letter too.

Impact: Implementations that use lowercase mapping, case folding or NFKC give different categories for the same marker. The fullwidth U+FF26 case flips the answer key with only a warning.

Suggested fix: Define the comparison: for example, NFKC then case folding before the category lookup, so U+FF26 is FALSE and ſ is TRUE. Or keep exact comparison of the letter and its simple lowercase, and make a fullwidth U+FF34/U+FF26 an error. Add corpus examples for the chosen rule.

F27: `marker` is 'exactly as it was written', but the `ws` inside the brackets is dropped

Location: docs/question-types/true-false.md:113, docs/question-types/true-false.md:137-139

```
| marker  | string  | The letter itself, exactly as it was written.          |
```

Description: The value rule allows `ws` around the letter (`[ T ]`, `[F ]`, `[\tT\t]`), and `examples/valid/true-false/spaced-markers.yaml` stores `marker: T` for each. The spec says `marker` is the letter 'exactly as it was written' and 'stored verbatim', and that it keeps the ability 'to be written back out unchanged' (line 142-143). It does not say that the spaces are removed, nor that spacing is lost on a round trip.

Impact: An implementation that reads 'verbatim' literally stores `T `, which then fails `malformed-true-false-marker` and the schema `maxLength: 1`.

Suggested fix: Say that `marker` is the letter of the value without the `ws` around it, and that only the letter and its case are kept.

F10: False-friend footnote speaks of an `S` 'meaning false'

Location: docs/question-types/true-false.md:256-257

Mechanical fix

```
[^4]: An `S` marker meaning false under `locale: id` SHOULD get
`false-friend-true-false-marker`, see [locale](#locale).
```

Description: An S never means false: it is always TRUE (line 83). The intent of the author cannot be known, and the rule (line 248) fires for every S under `id`. The footnote is also attached to the `locale-mismatched` row, where it does not apply.

Impact: A reader may think the lint needs to guess intent.

Suggested fix: 'Every S marker under `locale: id` SHOULD get `false-friend-true-false-marker`.' Attach the footnote only to that row.

F12: Type inference 'a value is a letter' does not say spaces are removed

Location: docs/question-types/question-base.md:230-234

```
3. a value is a letter: true/false;
```

Description: The true/false `value` rule allows ws around the letter (`[ T ]`), and `examples/valid/true-false/spaced-markers.mdq.md` infers true/false from such markers. Rule 3 says 'a value is a letter', and the value of `[ T ]` is `T `. 'letter' also does not link to its definition in true-false.md (one code point in `\p{L}`).

Impact: A literal reading infers multiple selection for `[ T ]` lists, then reports every item as `foreign-choice-marker`.

Suggested fix: 'a value, without the ws around it, is a [letter](true-false.md#body)'. Same for rules 1 and 2 (`[ x ]`, `[ * ]`).

F35: The introduction calls true statements 'correct answers'

Location: docs/question-types/true-false.md:5-7

Mechanical fix

```
rendition of a True or False marker. The correct answers are usually marked with
`[T]` and incorrect ones are usually marked with an `[F]`.
```

Description: In a true/false question every statement has a correct answer: `[F]` marks a false statement, and the correct answer for it is 'false'. 'Incorrect ones' suggests that `[F]` items are wrong answers, as `[ ]` items are in multiple choice. The Choices section uses the right terms ('true if the statement is true').

Impact: A reader can confuse the verdict of a statement with the correctness of a student's judgement, which the grading rules depend on.

Suggested fix: 'True statements are usually marked with `[T]` and false statements with `[F]`.'

F37: S is called the initial of words written in non-Latin scripts

Location: docs/question-types/true-false.md:87-88

Mechanical fix

```
`S` was chosen for TRUE over FALSE deliberately: it is the initial letter for
"true" in Hindi, Bengali, Arabic, Urdu and Japanese
```

Description: The cited words are in Devanagari, Bengali, Arabic, Urdu and Han script; S is the initial of their romanizations (satya, sahih, sach, shin), from the `ascii` column of the annex. The text does not say so. For Japanese, U+771F itself is already a TRUE letter.

Impact: Minor: the reasoning reads as wrong to a reader who knows these scripts.

Suggested fix: 'it is the initial of the romanized word for "true" in Hindi, Bengali, Arabic, Urdu and Japanese (satya, satya, sahih, sach, shin)'.

Omissions

Severity	Location	Description
MEDIUM	docs/responses.md:78-91	**F7** Responses do not say that a `null` statement value is an abstention
MEDIUM	docs/question-types/true-false.md:237-238, docs/question-types/true-false.md:107-144	**F26** The `marker/correct` agreement rule and the reserved X rule exist only in the table
LOW	docs/question-types/true-false.md:250	**F8** `uniform-true-false-answers` is not described in the prose

Severity	Location	Description
LOW	docs/question-types/true-false.md:141-144	**F39** No rule for the letter to write when `marker` is absent
LOW	docs/question-types/true-false.md:235	**F40** True/false corpus does not exercise some rules and edge cases
INFO	docs/question-types/true-false.md:83-85, docs/question-types/question-base.md:231-232	**F13** Cyrillic and Greek look-alikes of T, F and X are WARNING letters

F7: Responses do not say that a `null` statement value is an abstention

Location: docs/responses.md:78-91

```
A mapping from statement id to the judgement the student made. Unlike multiple
selection, an absent key is **not** a false: it is an abstention, and the
student may deliberately leave a statement unjudged.
```

Description: The response is 'a mapping of statement id to boolean' (line 29), and only an absent key is an abstention. `examples/grading/true-false.yaml:15-16` and its 'Explicit null' cases (line 68) treat `earth-is-flat: null` as an abstention too. The spec does not say whether `null` is an abstention or a malformed response, nor what a non-boolean value (`"T"`, 1) is.

Impact: An implementation that follows the spec text rejects a response that the shared grading corpus requires it to accept.

Suggested fix: In docs/responses.md, say that a `null` value is the same as an absent key, and that any other non-boolean value is a malformed response. Do the same for multiple selection if `null` is allowed there.

F26: The `marker`/`correct` agreement rule and the reserved `X` rule exist only in the table

Location: docs/question-types/true-false.md:237-238, docs/question-types/true-false.md:107-144

```
| choices[].marker   | error    | reserved-true-false-marker              | must not be `X` or
 `x`[^2]                                  |
| choices[].marker   | error    | true-false-marker-disagrees-with-correct | must agree with
`correct` according to its category[^3]    |
```

Description: The Choices section says a statement records the letter in two fields, but never states that they MUST agree (a FALSE letter with `correct: true`, or a TRUE or WARNING letter with `correct: false`). The prose also never says that a `marker` of X or x is an error: line 49 and line 85 only exclude X from the letters that have a truth value. Both rules are errors that appear only in the Additional Rules table. The WARNING case (`N` with `correct: false` is an error, `examples/invalid/model-only/true-false-marker-disagrees-warning.yaml`) is described only in `docs/lint-codes.md:88`.

Impact: A reader of the prose cannot know that a YAML document with `marker: F, correct: true` or `marker: X` is invalid, nor that a WARNING letter forces `correct: true`.

Suggested fix: Under Choices, add: 'When `marker` is present, its category MUST agree with `correct`: FALSE letters with `false`, TRUE and WARNING letters with `true` (`true-false-marker-disagrees-with-correct`). `marker` MUST NOT be X or x (`reserved-true-false-marker`).'

Exploitability conditions: Documents written in YAML or JSON, where `marker` and `correct` are independent.

F8: `uniform-true-false-answers` is not described in the prose

Location: docs/question-types/true-false.md:250

Mechanical fix

```
| choices[].correct  | info     | uniform-true-false-answers              | the statements are
 all true or all false                    |
```

Description: The Additional Rules table and `docs/lint-codes.md:51` list the info, but no section of true-false.md mentions it or gives it a MAY.

Impact: The rule exists only in the tables, without its rationale.

Suggested fix: Add a sentence, for example under Grading: 'Implementations MAY report, at the info level, a question whose statements are all true or all false (`uniform-true-false-answers`).'

F39: No rule for the letter to write when `marker` is absent

Location: docs/question-types/true-false.md:141-144

```
`marker` is OPTIONAL. A document carrying only `correct` is well-formed and
grades identically; it has simply lost the ability to be written back out
unchanged,
```

Description: Both implementations serialize questions to MDQ markdown. For a statement with only `correct` (`examples/valid/true-false/no-markers.yaml`), the spec does not say which letter a writer emits: T/F, or the letters of the `locale` row of the Locale table.

Impact: Two writers produce different markdown for the same document.

Suggested fix: State a default: 'A writer that needs a letter for a statement without `marker` SHOULD use the letters of the question's locale from the Locale table, and T/F otherwise.'

F40: True/false corpus does not exercise some rules and edge cases

Location: docs/question-types/true-false.md:235

```
| choices[]          | error    | foreign-choice-marker                   | must use a value
allowed in a true/false list              |
```

Description: No corpus example covers: `[x]` or `[TRUE]` in a markdown list with `type: true-false` (mdq-py gives `foreign-choice-marker`); a markdown list with a WARNING letter (`unlisted-marker` has only a YAML form); a list whose statements are all false (`uniform-true-false-answers` is tested only for all true); `all-or-nothing` with an abstention on a list whose size gives an inexact score (three statements); a `locale` with an unexpected TRUE letter that is not T (S under `pt`).

Impact: Implementations can disagree on these cases without a failing test.

Suggested fix: Add the examples once the related rules are settled.

F13: Cyrillic and Greek look-alikes of `T`, `F` and `X` are WARNING letters

Location: docs/question-types/true-false.md:83-85, docs/question-types/question-base.md:231-232

```
* **WARNING**: Every other letter, except `X` and `x`.
```

Description: Cyrillic `U+0422` (U+0422), `U+0425` (U+0425) and Greek `U+03A4` (U+03A4), `U+03A7` (U+03A7) look like T and X. A list of only Cyrillic `[U+0425]` items is inferred as true/false and every statement reads as true, with `unlisted-true-false-marker`. Russian 'U+0432U+0435U+0440U+043DU+043E' starts with Cyrillic `U+0412`, which looks like Latin B. The warning catches these, but its message cannot tell the author why.

Impact: Low: the warning fires, but the cause is invisible in most fonts.

Suggested fix: Optionally mention look-alike letters in the WARNING paragraph, so implementations can give a better message.

Inconsistencies

Severity	Location	Description
MEDIUM	docs/question-types/true-false.md:149, docs/question-types/true-false.md:247	**F4** Locale warning is a MAY in the prose and a warning in the table
MEDIUM	docs/question-types/true-false.md:236, schema/true-false.yaml:94-95, docs/lint-codes.md:84	**F28** The schema and `malformed-true-false-marker` both reject a marker that is not one code point
LOW	docs/question-types/true-false.md:183, docs/question-types/true-false.md:201-211, docs/question-types/true-false.md:218-225	**F9** Abstention is called skip, unmarked, missing and unjudged
LOW	docs/question-types/true-false.md:83	**F11** TRUE letters written as one code span TVS
LOW	docs/question-types/true-false.md:250	**F33** `uniform-true-false-answers` is attached to `choices[].correct`, but reported on `choices`
LOW	docs/question-types/true-false.md:250, docs/question-types/multiple-selection.md:161-162	**F34** True/false uses one code where choice questions use `no-correct-choice` and `all-choices-correct`
LOW	docs/question-types/true-false.md:78-81	**F36** The annex is said to hold a rationale that it does not have
LOW	docs/question-types/true-false.md:201-211	**F38** True/false grading does not define a correct marking, unlike multiple selection
LOW	examples/invalid/model-only/true-false-duplicate-choice-text.yaml:1-3	**F41** Corpus comment cites multiple-choice.md for the duplicate-text rule

F4: Locale warning is a MAY in the prose and a warning in the table

Location: docs/question-types/true-false.md:149, docs/question-types/true-false.md:247

```
Implementations MAY issue warnings for True/False letters that are incompatible
```

Description: The audit convention maps MAY to info, SHOULD to warning. The prose says implementations MAY issue `locale-mismatched-true-false-marker`, but the Additional Rules row (line 247) and `docs/lint-codes.md:28` give it the warning level. The same section uses SHOULD for `false-friend-true-false-marker` (line 172).

Impact: The requirement level of the rule is unclear.

Suggested fix: Change the prose to SHOULD (keep warning), or move the rule to info. SHOULD agrees with the corpus (`examples/valid/true-false/locale-mismatch.lint.json`).

F28: The schema and `malformed-true-false-marker` both reject a marker that is not one code point

Location: docs/question-types/true-false.md:236, schema/true-false.yaml:94-95, docs/lint-codes.md:84

```
| choices[].marker   | error    | malformed-true-false-marker             | must be a single
letter[^2]                                |
```

Description: The schema has `minLength: 1` and `maxLength: 1` on `marker`, so an empty, multi-character or combining-sequence marker is a schema violation. The Additional Rules table lists the same check as `malformed-true-false-marker` without '(schema)', and `docs/question-types/question-base.md:329` says the table lists only rules the schema does not capture. `docs/lint-codes.md:84` says `malformed-true-false-marker` covers an empty or longer marker. `examples/invalid/true-false-combining-mark-marker.yaml:3` says the schema 'rejects it too', but its `.lint.json` expects only `malformed-true-false-marker`, and mdq-py reports no `schema-error` for `marker: ""` or `marker: "TRUE"`. The spec does not say which code wins when both apply.

Impact: An implementation that reports schema errors as `schema-error` fails the corpus for these documents, or reports two errors for one defect.

Suggested fix: Either remove `minLength`/`maxLength` from the schema and leave the check to `malformed-true-false-marker`, or state that a dedicated code replaces `schema-error` when both apply to the same field.

F9: Abstention is called skip, unmarked, missing and unjudged

Location: docs/question-types/true-false.md:183, docs/question-types/true-false.md:201-211, docs/question-types/true-false.md:218-225

Mechanical fix

```
Including abstentions is deliberate: a student who skips a statement because
```

Description: GLOSSARY.md defines Abstention and says to avoid 'skip' for it; Skip is the absence of a response to a whole question. Line 183 says 'skips a statement'. The Grading section uses 'missing marking' (201), 'abstain' (205), 'unmarked' (210-211), and the table uses _ without saying what it is.

Impact: 'skips a statement' collides with the defined term Skip, which has different grading (no feedback).

Suggested fix: Use 'abstains on a statement' / 'abstention' throughout, and say in the table intro that _ is an abstention.

F11: TRUE letters written as one code span TVS

Location: docs/question-types/true-false.md:83

Mechanical fix

```
* **TRUE**: `TVS`, plus
```

Description: TVS reads as a three-letter string. The CJK letters on the same line are not in code spans.

Impact: Minor: a reader may take TVS as one marker.

Suggested fix: T, V, S, U+5BF9 and U+771F; the same for the FALSE line.

F33: `uniform-true-false-answers` is attached to `choices[].correct`, but reported on `choices`

Location: docs/question-types/true-false.md:250

Mechanical fix

```
| choices[].correct  | info     | uniform-true-false-answers              | the statements are
 all true or all false                    |
```

Description: The rule is about the whole list. `examples/valid/true-false/unlisted-marker.lint.json` and mdq-py report it at the path `choices`, and the matching multiple-selection rules (`no-correct-choice`, `all-choices-correct`, multiple-selection.md:161-162) use the field `choices`.

Impact: An implementation that follows the table reports the info on each statement instead of once on the list, and fails the corpus.

Suggested fix: Change the Field column to `choices`.

F34: True/false uses one code where choice questions use `no-correct-choice` and `all-choices-correct`

Location: docs/question-types/true-false.md:250, docs/question-types/multiple-selection.md:161-162

```
| choices[].correct  | info     | uniform-true-false-answers              | the statements are
 all true or all false                    |
```

Description: Multiple selection reports a list with no `correct: true` as `no-correct-choice` and a list with every `correct: true` as `all-choices-correct`. True/false has the same field `correct` and the same two cases, but reports both as `uniform-true-false-answers`.

Impact: A tool that filters diagnostics by code needs a second code for the same property of the same field.

Suggested fix: Reuse `no-correct-choice` and `all-choices-correct` for true/false, or document why true/false has its own code.

F36: The annex is said to hold a rationale that it does not have

Location: docs/question-types/true-false.md:78-81

Mechanical fix

```
[annex/true-false-spellings.md](../annex/true-false-spellings.md)
for the source data and the rationale behind each addition.
```

Description: `docs/annex/true-false-spellings.md` contains only the spelling table and the references. The rationale for the letter assignments (S for TRUE, the letters left in WARNING) is in true-false.md:87-94.

Impact: A reader who follows the link does not find the rationale.

Suggested fix: 'for the source data. The rationale follows below.'

F38: True/false grading does not define a correct marking, unlike multiple selection

Location: docs/question-types/true-false.md:201-211

```
* **partial**: Each correct marking gives a point, and each incorrect or missing
  marking is ignored.
```

Description: multiple-selection.md:107-110 defines 'judged correctly' before the strategies. True/false uses 'correct marking', 'correctly marked', 'incorrect marking' and 'mistake' without saying that a marking is the boolean of the response for a statement and that it is correct when it equals the statement's `correct`. The same property gets different terms in the two documents.

Impact: A reader must infer how a response value is compared with the answer key.

Suggested fix: Before the list: 'A statement is **judged correctly** when the response holds its `correct` value, and **judged wrongly** when it holds the other value. An abstention is neither.' Then use these terms in the three strategies.

F41: Corpus comment cites multiple-choice.md for the duplicate-text rule

Location: examples/invalid/model-only/true-false-duplicate-choice-text.yaml:1-3

Mechanical fix

```
# are compared with whitespace normalized, so this is `duplicate-choice-text`
# (multiple-choice.md, "Choices"), which JSON Schema cannot express.
```

Description: The duplicate-text rule is in `docs/references/choices.md`, section 'Choice text' (lines 115-119). multiple-choice.md 'Choices' only links to it.

Impact: Minor: the reference is out of date.

Suggested fix: Cite `references/choices.md, "Choice text"`.

Grammar violations

Severity	Location	Description
MEDIUM	docs/question-types/true-false.md:43-50	**F15** The `true` and `false` rules are never defined, and WARNING letters have no rule
MEDIUM	docs/question-types/true-false.md:52-57	**F16** `\p{L}` breaks the regex dialect rule of the common grammar
MEDIUM	docs/question-types/true-false.md:52-57	**F17** The Unicode version of the letter test is not fixed, so Python and JavaScript disagree
MEDIUM	docs/references/choices.md:56	**F25** An item with other block content fails the `item` rule, and no error code is given
LOW	docs/question-types/true-false.md:52-53	**F20** `\p{L}` includes signs that do not look like letters
LOW	docs/question-types/true-false.md:235-237	**F24** Two error codes apply to `[x]` and `[TF]` in a list forced to true/false
INFO	docs/question-types/true-false.md:65-68	**F23** Some valid single-code-point markers are not one code point after NFC

F15: The `true` and `false` rules are never defined, and WARNING letters have no rule

Location: docs/question-types/true-false.md:43-50

```
value : ws? (true | false) ws?
```

Description: No snippet defines `true` or `false`; `scripts/spec_audit.py` compiles the snippet only with stubs. The letter sets exist only in prose (lines 83-98), and the WARNING category, which means true, is not visible in the rule. A reader cannot tell from the grammar that `[Q]` matches `true`. Testing needed a hand-built grammar: `true : /[TtVvSsU+5BF9U+771F]/`, `false : /[FfU+9519U+507D]/`, and a third alternative for every other letter except X/x.

Impact: The grammar does not decide which markers are valid; implementations rebuild the sets from prose.

Suggested fix: Define the rules in the snippet, for example `true : TRUE_LETTER | WARNING_LETTER`, `false : FALSE_LETTER`, with terminals for each set, and explain `WARNING_LETTER` as a letter outside the other sets and outside X/x.

F16: `\p{L}` breaks the regex dialect rule of the common grammar

Location: docs/question-types/true-false.md:52-57

```
A **letter** is exactly one Unicode code point whose General_Category is one of
`Lu`, `Ll`, `Lt`, `Lm` or `Lo` -- that is, the character class written `\p{L}`
in regex engines that support Unicode properties.
```

Description: grammar.md ("Regex dialect") says terminals use only explicit character classes that match the same characters in Python `re`, JavaScript RegExp and JSON Schema `pattern`. `\p{L}` does not meet this: `re.compile(r'\p{L}')` raises `bad escape \p` in Python 3.13; in JavaScript without the u flag, `/^\p{L}$/` is a literal p{L} and does not match p or U+771F; without u, `/^[^\]]$/` does not match U+20000 (two UTF-16 units). Lark's default `re` backend cannot use it either; the explicit class built from `unicodedata` for the test is about 10 KB.

Impact: The letter terminal cannot be written in the dialect the spec mandates, so each implementation picks its own engine and flags.

Suggested fix: State the exception in grammar.md: the letter test uses the Unicode General_Category of one code point (Python `unicodedata.category`, JavaScript `/^\p{L}$/u`), not a regex of the common dialect.

F17: The Unicode version of the letter test is not fixed, so Python and JavaScript disagree

Location: docs/question-types/true-false.md:52-57

```
A **letter** is exactly one Unicode code point whose General_Category is one of
`Lu`, `Ll`, `Lt`, `Lm` or `Lo`
```

Description: General_Category changes between Unicode versions. Python 3.13 here has Unicode 15.1; Node has Unicode 17.0. U+10D50 GARAY CAPITAL LETTER A (Unicode 16.0) is `Cn` in Python and matches `/^\p{L}$/u` in Node. Counter-example: `* [\u{10D50}] a` / `* [F] b` is a valid true/false list in mdq-js and, in mdq-py, rule 4 of type inference does not apply because rule 3 does (F), but the first value is a `foreign-choice-marker`. A list of only such letters is true/false in one implementation and multiple selection in the other.

Impact: The same document gets a different type or different errors in each implementation, which grammar.md explicitly tries to prevent.

Suggested fix: Fix a minimum Unicode version for the letter test, or state that letters unassigned in that version are not letters.

F25: An item with other block content fails the `item` rule, and no error code is given

Location: docs/references/choices.md:56

```
item      : ws? "[" value VALUE_END ws? (CHOICE_ID ws?)? md_inline* nl lazy_line* (blank_line*
observations)?
```

Description: The rule allows one paragraph of text and then the observations. CommonMark gives an item more blocks, and these inputs fail the rule in the test: a second paragraph (`* [F] Flat` / blank / `More text.`), a nested list (`* [T] a` / `* sub`), a fenced code block, and text after the feedback (`> fb` / blank / `Later text.`). The spec states `parse-error` for a second feedback block, but not for these. It does not say if the list stops being a choice list, if the item is a `parse-error`, or if the extra blocks join the text.

Impact: Implementations can treat a common authoring mistake differently.

Suggested fix: State the result: for example, any block in a choice item other than the first paragraph, the feedback and the comment is a `parse-error`.

F20: `\p{L}` includes signs that do not look like letters

Location: docs/question-types/true-false.md:52-53

```
A **letter** is exactly one Unicode code point whose General_Category is one of
`Lu`, `Ll`, `Lt`, `Lm` or `Lo`
```

Description: `Lm` and `Lo` contain characters that read as punctuation or symbols: ˆ (U+02C6), U+30FC (U+30FC KATAKANA-HIRAGANA PROLONGED SOUND MARK, looks like a dash), U+3005 (U+3005), and the ordinal indicators ª (U+00AA) and º (U+00BA), common in pt-BR. Counter-example: `* [U+30FC] a` is accepted as a WARNING letter and means true, while the spec excludes ✓, × and U+3007 because they are not letters.

Impact: The goal of the exclusion ("marks that look like answers but are not letters are NOT valid") is only partly met.

Suggested fix: Restrict the WARNING set to `Lu`, `Ll`, `Lt` and `Lo` minus a short exclusion list, or accept the behaviour and list these examples.

F24: Two error codes apply to `[x]` and `[TF]` in a list forced to true/false

Location: docs/question-types/true-false.md:235-237

```
| choices[]          | error    | foreign-choice-marker                   | must use a value
allowed in a true/false list              |
| choices[].marker   | error    | malformed-true-false-marker             | must be a single
letter[^2]                                |
| choices[].marker   | error    | reserved-true-false-marker              | must not be `X` or
 `x`[^2]                                  |
```

Description: With `type: true-false`, `* [x] a` fails the `value` rule (`foreign-choice-marker`) and also has a marker `x` (`reserved-true-false-marker`); `* [TF] a` and `* [✓] a` match both `foreign-choice-marker` and `malformed-true-false-marker`. The table does not say which one a markdown document gets. The corpus uses `foreign-choice-marker` for markdown (`examples/invalid/true-false-symbol-marker`) and the other two only for YAML (`examples/invalid/model-only/`), and `docs/lint-codes.md:84` ties them to the `marker` field, but true-false.md does not state this.

Impact: Implementations can report different codes for the same markdown input.

Suggested fix: Say in the table that `malformed-true-false-marker` and `reserved-true-false-marker` apply to the `marker` field of model input, and that a markdown value outside the `value` rule is `foreign-choice-marker`.

F23: Some valid single-code-point markers are not one code point after NFC

Location: docs/question-types/true-false.md:65-68

```
* "One code point", not one grapheme cluster. A base letter followed by a
  combining mark is two code points and is not a valid marker.
```

Description: U+0915U+093C U+0958 DEVANAGARI LETTER QA is one Lo code point, so * [U+0915U+093C] a is a valid WARNING marker (accepted in the test). Its NFC form is two code points (U+0915 U+093C, a composition exclusion). An editor or pipeline that normalizes to NFC turns the valid marker into a `foreign-choice-marker`. Conversely, [é] is valid as U+00E9 and invalid as e + U+0301. The spec does not say whether the parser normalizes the source.

Impact: Validity of a marker can change when an editor normalizes the file.

Suggested fix: State that the parser does not normalize the source, and mention composition exclusions next to the NFC lint.

Schema mismatches

Severity	Location	Description
LOW	schema/true-false.yaml:86-95	**F29** Schema accepts any single code point as `marker`
LOW	schema/true-false.yaml:88-93	**F30** Schema `marker` description omits the agreement check
LOW	schema/true-false.yaml:33-37	**F31** Schema `grading` description ignores the exam's grading map

F29: Schema accepts any single code point as `marker`

Location: schema/true-false.yaml:86-95

```
      marker:
        type: string
```

Description: The spec defines `marker` as one code point in `\p{L}`, other than X or x (true-false.md:52-64, 236-237). The schema checks only the length, so `marker: "1"`, `marker: "✓"` and `marker: X` pass it. Its description says 'The single letter', which the schema does not enforce.

Impact: A consumer that relies on the schema alone accepts markers the spec forbids.

Suggested fix: Add `pattern: "^\\p{L}$"` and `not: {enum: [X, x]}`, or say in the description that the letter and reserved checks are model rules (`malformed-true-false-marker`, `reserved-true-false-marker`). The first option interacts with the code-precedence question of the length check.

F30: Schema `marker` description omits the agreement check

Location: schema/true-false.yaml:88-93

Mechanical fix

```
carries the meaning, but the letter itself is needed to warn
          about WARNING letters and locale mismatches, and to
          round-trip the document unchanged.
```

Description: The description lists the uses of `marker` but omits the error `true-false-marker-disagrees-with-correct`, which needs it, and `false-friend-true-false-marker`. It does not say that a marker's category must agree with `correct`.

Impact: A producer that reads only the schema may write a `marker` that contradicts `correct`.

Suggested fix: Add: 'When present, its category must agree with `correct`.'

F31: Schema `grading` description ignores the exam's grading map

Location: schema/true-false.yaml:33-37

Mechanical fix

```
`inherit`, the default, takes the exam's `grading` value, and
          is `symmetric` outside an exam.
```

Description: The exam's `grading` can be a map from question type to strategy (docs/exam.md:333-348); a question then takes the `true-false` entry, or `symmetric` when the key is missing. true-false.md:194-199 says so; the schema description says only 'the exam's `grading` value'.

Impact: A reader of the schema may think a map value is copied into the question.

Suggested fix: 'takes the exam's `grading` (its `true-false` entry when it is a map), and is `symmetric` outside an exam or when the exam sets none.'

Prioritized recommendations

P1: Make true/false all-or-nothing agree with the glossary and the other types. (Findings: F1)

P1: Define the letter test formally: letter terminals, Unicode version, case mapping and the NFC info. (Findings: F5, F6, F15, F16, F17, F20)

P2: Settle the locale and marker rules so each case has one code and one level, in prose and table. (Findings: F2, F3, F4, F24, F26, F27, F28, F29, F39)

P2: Close the shared gaps: null in responses, extra blocks in choice items, the grading terms and the uniform-answers code. (Findings: F7, F12, F25, F34, F38)

P3: Apply the mechanical fixes in one pass and extend the corpus. (Findings: F8, F9, F10, F11, F30, F31, F32, F33, F35, F36, F37, F40, F41)

GitHub issues

Each block below is a complete issue, ready to paste.

````
--- ISSUE 1 ---
# [Spec] True/false all-or-nothing must not give partial credit

**Labels:** high

## Problem

True/false `all-or-nothing` scores `correctly marked / total` when there is no mistake, so
abstentions still give partial credit. Options: (a) recommended: 1 only when every statement is
 judged correctly, an abstention counts as a mistake, as in the glossary, multiple selection
and fill-in; (b) keep the rule under a new strategy name and update the glossary and exam.md.

## Evidence

- `docs/question-types/true-false.md:204-207`: True/false all-or-nothing gives partial credit

  ```markdown
  * **all-or-nothing**: The student must mark all choices with the correct marking
    or abstain. Any mistake in the marked choices gives zero points. If all marked
    choices are correct, the student gets a score proportional to the number of
    correct markings (`correctly marked / total number of choices`).
  ```

## Impact

An exam that sets `grading: all-or-nothing` for every type gets two different meanings. An
instructor who expects no credit for an incomplete true/false answer gives 0.5 to a student who
 judged half of the statements.

## Suggested fix

Either make true/false all-or-nothing score 1 only when every statement is judged correctly (an
 abstention counts as a mistake), or keep the current rule, give it a different name, and
update the glossary. The first option agrees with the glossary and the other types.

## Acceptance criteria

- [ ] true-false.md all-or-nothing agrees with GLOSSARY.md All-or-nothing
- [ ] The grading table and examples/grading/true-false.yaml agree with the new rule
- [ ] scripts/spec_audit.py passes

--- END ISSUE 1 ---
````

````
--- ISSUE 2 ---
# [Spec] Define the letter test of true/false markers

**Labels:** medium, low

## Problem

The letter sets exist only in prose, `\p{L}` is outside the regex dialect of grammar.md, the
Unicode version is not fixed, the case mapping is not defined (fullwidth `U+FF26` reads as
true), `Lm` admits signs such as `U+30FC`, and the NFC info mixes combining sequences with
singletons. Recommended: define `TRUE_LETTER`, `FALSE_LETTER` and `WARNING_LETTER` in the
snippet; state that the letter test uses General_Category of one code point from a fixed
minimum Unicode version; compare after NFKC and case folding (so `U+FF26` is FALSE); restrict
WARNING to `Lu`, `Ll`, `Lt`, `Lo`; reword the NFC info for singletons only.

## Evidence

- `docs/question-types/true-false.md:65-68, docs/question-types/true-false.md:251`: NFC rule
mixes combining sequences with non-NFC single code points

  ```markdown
  * "One code point", not one grapheme cluster. A base letter followed by a
    combining mark is two code points and is not a valid marker. Implementations
    MAY report, at the info level, a marker that is not in NFC form: in NFC,
    precomposed forms are single code points.
  ```
- `docs/question-types/true-false.md:46-50, docs/question-types/true-false.md:83-86,
docs/question-types/true-false.md:96-98`: Case-insensitivity and compatibility forms of letters
 are not defined

  ```markdown
  We only show the upper case versions of each symbol. Each set must be expanded
  with lowercase versions as well; the CJK characters have no case and are used
  as-is.
  ```
- `docs/question-types/true-false.md:43-50`: The `true` and `false` rules are never defined,
and WARNING letters have no rule

  ```lark
  value : ws? (true | false) ws?
  ```
- `docs/question-types/true-false.md:52-57`: `\p{L}` breaks the regex dialect rule of the
common grammar

  ```markdown
  A **letter** is exactly one Unicode code point whose General_Category is one of
  `Lu`, `Ll`, `Lt`, `Lm` or `Lo` -- that is, the character class written `\p{L}`
  in regex engines that support Unicode properties.
  ```
- `docs/question-types/true-false.md:52-57`: The Unicode version of the letter test is not
fixed, so Python and JavaScript disagree

  ```markdown
  A **letter** is exactly one Unicode code point whose General_Category is one of
  `Lu`, `Ll`, `Lt`, `Lm` or `Lo`
  ```
- `docs/question-types/true-false.md:52-53`: `\p{L}` includes signs that do not look like
letters

  ```markdown
  A **letter** is exactly one Unicode code point whose General_Category is one of
  `Lu`, `Ll`, `Lt`, `Lm` or `Lo`
  ```

## Impact

- **NFC rule mixes combining sequences with non-NFC single code points:** An implementer can
emit the info for combining sequences, or miss the singleton case.
- **Case-insensitivity and compatibility forms of letters are not defined:** Implementations
that use lowercase mapping, case folding or NFKC give different categories for the same marker.
````

```
 The fullwidth `U+FF26` case flips the answer key with only a warning.
- **The `true` and `false` rules are never defined, and WARNING letters have no rule:** The
grammar does not decide which markers are valid; implementations rebuild the sets from prose.
- **`\p{L}` breaks the regex dialect rule of the common grammar:** The letter terminal cannot
be written in the dialect the spec mandates, so each implementation picks its own engine and
flags.
- **The Unicode version of the letter test is not fixed, so Python and JavaScript disagree:**
The same document gets a different type or different errors in each implementation, which
grammar.md explicitly tries to prevent.
- **`\p{L}` includes signs that do not look like letters:** The goal of the exclusion ("marks
that look like answers but are not letters are NOT valid") is only partly met.

## Suggested fix

- **NFC rule mixes combining sequences with non-NFC single code points:** Split the bullet: a
combining sequence is two code points and is `malformed-true-false-marker`; a single code point
 that NFC changes (U+212B to U+00C5) MAY get `non-nfc-true-false-marker`. Reword the table row
to 'is a single code point that NFC changes'.
- **Case-insensitivity and compatibility forms of letters are not defined:** Define the
comparison: for example, NFKC then case folding before the category lookup, so `U+FF26` is
FALSE and `ſ` is TRUE. Or keep exact comparison of the letter and its simple lowercase, and
make a fullwidth `U+FF34`/`U+FF26` an error. Add corpus examples for the chosen rule.
- **The `true` and `false` rules are never defined, and WARNING letters have no rule:** Define
the rules in the snippet, for example `true : TRUE_LETTER | WARNING_LETTER`, `false :
FALSE_LETTER`, with terminals for each set, and explain `WARNING_LETTER` as a letter outside
the other sets and outside `X`/`x`.
- **`\p{L}` breaks the regex dialect rule of the common grammar:** State the exception in
grammar.md: the letter test uses the Unicode General_Category of one code point (Python
`unicodedata.category`, JavaScript `/^\p{L}$/u`), not a regex of the common dialect.
- **The Unicode version of the letter test is not fixed, so Python and JavaScript disagree:**
Fix a minimum Unicode version for the letter test, or state that letters unassigned in that
version are not letters.
- **`\p{L}` includes signs that do not look like letters:** Restrict the WARNING set to `Lu`,
`Ll`, `Lt` and `Lo` minus a short exclusion list, or accept the behaviour and list these
examples.

## Acceptance criteria

- [ ] The `value` snippet defines `true` and `false`
- [ ] grammar.md states the exception to the regex dialect for the letter test
- [ ] The spec fixes the Unicode version and the case mapping
- [ ] Corpus examples cover fullwidth and long-s markers
- [ ] scripts/spec_audit.py passes

--- END ISSUE 2 ---
```

````
--- ISSUE 3 ---
# [Spec] Define the locale mismatch rule

**Labels:** medium

## Problem

Only one example defines a locale mismatch, the text refers to an absent `id` row, and the
prose says MAY for a warning. Recommended: a TRUE or FALSE letter whose folded form is not the
expected letter of its category is mismatched; WARNING letters get only `unlisted-true-false-
marker`; the prose says SHOULD; delete the `id` row sentence. This is what mdq-py does.

## Evidence

- `docs/question-types/true-false.md:173-175`: Locale text refers to an `id` row that the table
 does not have

  ```markdown
  dedicated warning, `false-friend-true-false-marker`. The `id` row has no
  mismatched form, so `locale-mismatched-true-false-marker` is never issued for
  it.
  ```
- `docs/question-types/true-false.md:149-153`: The spec does not define which markers are
locale-mismatched

  ```markdown
  Implementations MAY issue warnings for True/False letters that are incompatible
  with the locale for the question. For instance, if `locale: pt-BR`, it may
  expect `V` and `F` -- for "verdadeiro" (true) and "falso" (false) -- and could
  issue a warning if `T` is used instead.
  ```
- `docs/question-types/true-false.md:149, docs/question-types/true-false.md:247`: Locale
warning is a MAY in the prose and a warning in the table

  ```markdown
  Implementations MAY issue warnings for True/False letters that are incompatible
  ```

## Impact

- **Locale text refers to an `id` row that the table does not have:** A reader cannot tell
whether `id` is meant to be in the table (with which letters) or not.
- **The spec does not define which markers are locale-mismatched:** Two implementations can
report different warnings for the same document.
- **Locale warning is a MAY in the prose and a warning in the table:** The requirement level of
 the rule is unclear.

## Suggested fix

- **Locale text refers to an `id` row that the table does not have:** Delete the sentence about
 the `id` row, or add an `id` row (`B` / `S`? `T` / `F`?) and define what 'no mismatched form'
means.
- **The spec does not define which markers are locale-mismatched:** State the rule: under a
locale in the table, a marker in the TRUE or FALSE category whose case-folded form is not the
expected letter of its category is mismatched. WARNING letters get only `unlisted-true-false-
marker`.
- **Locale warning is a MAY in the prose and a warning in the table:** Change the prose to
SHOULD (keep warning), or move the rule to info. SHOULD agrees with the corpus
(`examples/valid/true-false/locale-mismatch.lint.json`).

## Acceptance criteria

- [ ] true-false.md states which markers are mismatched
- [ ] The level in prose and table agree
- [ ] No reference to a missing `id` row
- [ ] A corpus example covers `S` under `pt`

--- END ISSUE 3 ---
````

````
--- ISSUE 4 ---
# [Spec] Settle the rules of the `marker` field

**Labels:** medium, low

## Problem

The agreement and reserved-`X` rules are only in the table; `malformed-` and `reserved-true-
false-marker` overlap with `foreign-choice-marker` in markdown and with `schema-error` in YAML;
 `marker` is called verbatim but loses the spaces; the schema checks only length; no default
letter for writers. Recommended: describe the rules under Choices; say the two marker codes
apply to model input and markdown values outside `value` are `foreign-choice-marker`; say a
dedicated code replaces `schema-error` for the same field (keep `maxLength`); `marker` is the
letter without `ws`; writers use the locale letters, else `T`/`F`.

## Evidence

- `docs/question-types/true-false.md:235-237`: Two error codes apply to `[x]` and `[TF]` in a
list forced to true/false

  ```markdown
  | choices[]          | error    | foreign-choice-marker                    | must use a value
 allowed in a true/false list              |
  | choices[].marker   | error    | malformed-true-false-marker              | must be a single
 letter[^2]                                |
  | choices[].marker   | error    | reserved-true-false-marker               | must not be `X`
or `x`[^2]                                 |
  ```
- `docs/question-types/true-false.md:237-238, docs/question-types/true-false.md:107-144`: The
`marker`/`correct` agreement rule and the reserved `X` rule exist only in the table

  ```markdown
  | choices[].marker   | error    | reserved-true-false-marker               | must not be `X`
or `x`[^2]                                 |
  | choices[].marker   | error    | true-false-marker-disagrees-with-correct | must agree with
`correct` according to its category[^3]    |
  ```
- `docs/question-types/true-false.md:113, docs/question-types/true-false.md:137-139`: `marker`
is 'exactly as it was written', but the `ws` inside the brackets is dropped

  ```markdown
  | marker  | string  | The letter itself, exactly as it was written.           |
  ```
- `docs/question-types/true-false.md:236, schema/true-false.yaml:94-95, docs/lint-codes.md:84`:
 The schema and `malformed-true-false-marker` both reject a marker that is not one code point

  ```markdown
  | choices[].marker   | error    | malformed-true-false-marker              | must be a single
 letter[^2]                                |
  ```
- `schema/true-false.yaml:86-95`: Schema accepts any single code point as `marker`

  ```yaml
        marker:
          type: string
  ```
- `docs/question-types/true-false.md:141-144`: No rule for the letter to write when `marker` is
 absent

  ```markdown
  `marker` is OPTIONAL. A document carrying only `correct` is well-formed and
  grades identically; it has simply lost the ability to be written back out
  unchanged,
  ```

## Impact

- **Two error codes apply to `[x]` and `[TF]` in a list forced to true/false:** Implementations
 can report different codes for the same markdown input.
- **The `marker`/`correct` agreement rule and the reserved `X` rule exist only in the table:**
````

```
A reader of the prose cannot know that a YAML document with `marker: F, correct: true` or
`marker: X` is invalid, nor that a WARNING letter forces `correct: true`.
- **`marker` is 'exactly as it was written', but the `ws` inside the brackets is dropped:** An
implementation that reads 'verbatim' literally stores ` T `, which then fails `malformed-true-
false-marker` and the schema `maxLength: 1`.
- **The schema and `malformed-true-false-marker` both reject a marker that is not one code
point:** An implementation that reports schema errors as `schema-error` fails the corpus for
these documents, or reports two errors for one defect.
- **Schema accepts any single code point as `marker`:** A consumer that relies on the schema
alone accepts markers the spec forbids.
- **No rule for the letter to write when `marker` is absent:** Two writers produce different
markdown for the same document.

## Suggested fix

- **Two error codes apply to `[x]` and `[TF]` in a list forced to true/false:** Say in the
table that `malformed-true-false-marker` and `reserved-true-false-marker` apply to the `marker`
 field of model input, and that a markdown value outside the `value` rule is `foreign-choice-
marker`.
- **The `marker`/`correct` agreement rule and the reserved `X` rule exist only in the table:**
Under Choices, add: 'When `marker` is present, its category MUST agree with `correct`: FALSE
letters with `false`, TRUE and WARNING letters with `true` (`true-false-marker-disagrees-with-
correct`). `marker` MUST NOT be `X` or `x` (`reserved-true-false-marker`).'
- **`marker` is 'exactly as it was written', but the `ws` inside the brackets is dropped:** Say
 that `marker` is the letter of the value without the `ws` around it, and that only the letter
and its case are kept.
- **The schema and `malformed-true-false-marker` both reject a marker that is not one code
point:** Either remove `minLength`/`maxLength` from the schema and leave the check to
`malformed-true-false-marker`, or state that a dedicated code replaces `schema-error` when both
 apply to the same field.
- **Schema accepts any single code point as `marker`:** Add `pattern: "^\\p{L}$"` and `not:
{enum: [X, x]}`, or say in the description that the letter and reserved checks are model rules
(`malformed-true-false-marker`, `reserved-true-false-marker`). The first option interacts with
the code-precedence question of the length check.
- **No rule for the letter to write when `marker` is absent:** State a default: 'A writer that
needs a letter for a statement without `marker` SHOULD use the letters of the question's locale
 from the Locale table, and `T`/`F` otherwise.'

## Acceptance criteria

- [ ] Each marker rule is in prose and table with one code per case
- [ ] The schema description lists the model rules
- [ ] scripts/spec_audit.py passes

--- END ISSUE 4 ---
```

````
--- ISSUE 5 ---
# [Spec] Reuse the choice-list codes for uniform true/false answers

**Labels:** low

## Problem

True/false reports both 'all true' and 'all false' as `uniform-true-false-answers`, while
multiple selection uses `no-correct-choice` and `all-choices-correct` for the same field.
Options: (a) reuse the two codes; (b) recommended: keep the true/false code, since 'all false'
is a valid key in true/false and the reason differs, and say so in the prose.

## Evidence

- `docs/question-types/true-false.md:250, docs/question-types/multiple-selection.md:161-162`:
True/false uses one code where choice questions use `no-correct-choice` and `all-choices-
correct`

  ```markdown
  | choices[].correct  | info     | uniform-true-false-answers               | the statements
are all true or all false                   |
  ```

## Impact

A tool that filters diagnostics by code needs a second code for the same property of the same
field.

## Suggested fix

Reuse `no-correct-choice` and `all-choices-correct` for true/false, or document why true/false
has its own code.

## Acceptance criteria

- [ ] true-false.md explains the code or reuses the multiple-selection ones
- [ ] lint-codes.md agrees

--- END ISSUE 5 ---
````

````
--- ISSUE 6 ---
# [Spec] A `null` statement value in a true/false response is an abstention

**Labels:** medium

## Problem

The grading corpus treats `null` as an abstention; docs/responses.md does not. Recommended: say
 `null` equals an absent key and any other non-boolean is a malformed response.

## Evidence

- `docs/responses.md:78-91`: Responses do not say that a `null` statement value is an
abstention

  ```markdown
  A mapping from statement id to the judgement the student made. Unlike multiple
  selection, an absent key is **not** a false: it is an abstention, and the
  student may deliberately leave a statement unjudged.
  ```

## Impact

An implementation that follows the spec text rejects a response that the shared grading corpus
requires it to accept.

## Suggested fix

In docs/responses.md, say that a `null` value is the same as an absent key, and that any other
non-boolean value is a malformed response. Do the same for multiple selection if `null` is
allowed there.

## Acceptance criteria

- [ ] docs/responses.md states the rule for true/false (and multiple selection)

--- END ISSUE 6 ---
````

````
--- ISSUE 7 ---
# [Spec] State the error for extra blocks in a choice item

**Labels:** medium

## Problem

A second paragraph, a nested list, a code block or text after the feedback fails the `item`
rule, and the spec gives no code. Recommended: `parse-error`, which mdq-py already does.

## Evidence

- `docs/references/choices.md:56`: An item with other block content fails the `item` rule, and
no error code is given

  ```lark
  item      : ws? "[" value VALUE_END ws? (CHOICE_ID ws?)? md_inline* nl lazy_line*
(blank_line* observations)?
  ```

## Impact

Implementations can treat a common authoring mistake differently.

## Suggested fix

State the result: for example, any block in a choice item other than the first paragraph, the
feedback and the comment is a `parse-error`.

## Acceptance criteria

- [ ] docs/references/choices.md gives the code
- [ ] An invalid corpus example covers it

--- END ISSUE 7 ---
````

````
--- ISSUE 8 ---
# [Spec] Define judged correctly and the value without spaces

**Labels:** low

## Problem

True/false grading uses 'correct marking' where multiple selection defines 'judged correctly';
type inference says 'a value is a letter' when the value can carry spaces. Recommended: define
judged correctly / wrongly / abstention before the strategies, and say inference reads the
value without `ws`.

## Evidence

- `docs/question-types/question-base.md:230-234`: Type inference 'a value is a letter' does not
 say spaces are removed

  ```markdown
  3. a value is a letter: true/false;
  ```
- `docs/question-types/true-false.md:201-211`: True/false grading does not define a correct
marking, unlike multiple selection

  ```markdown
  * **partial**: Each correct marking gives a point, and each incorrect or missing
    marking is ignored.
  ```

## Impact

- **Type inference 'a value is a letter' does not say spaces are removed:** A literal reading
infers multiple selection for `[ T ]` lists, then reports every item as `foreign-choice-
marker`.
- **True/false grading does not define a correct marking, unlike multiple selection:** A reader
 must infer how a response value is compared with the answer key.

## Suggested fix

- **Type inference 'a value is a letter' does not say spaces are removed:** 'a value, without
the `ws` around it, is a [letter](true-false.md#body)'. Same for rules 1 and 2 (`[ x ]`, `[ *
]`).
- **True/false grading does not define a correct marking, unlike multiple selection:** Before
the list: 'A statement is **judged correctly** when the response holds its `correct` value, and
 **judged wrongly** when it holds the other value. An abstention is neither.' Then use these
terms in the three strategies.

## Acceptance criteria

- [ ] true-false.md Grading defines the terms
- [ ] question-base.md inference trims the value

--- END ISSUE 8 ---
````

````
--- ISSUE 9 ---
# [Spec] Cover the remaining true/false edge cases in the corpus

**Labels:** low

## Problem

Add the examples listed in the finding once the rules above are settled.

## Evidence

- `docs/question-types/true-false.md:235`: True/false corpus does not exercise some rules and
edge cases

  ```markdown
  | choices[]          | error    | foreign-choice-marker                    | must use a value
 allowed in a true/false list              |
  ```

## Impact

Implementations can disagree on these cases without a failing test.

## Suggested fix

Add the examples once the related rules are settled.

## Acceptance criteria

- [ ] Each case in F40 has a corpus example
- [ ] scripts/spec_audit.py passes

--- END ISSUE 9 ---
````

````
--- ISSUE 10 ---
# Mechanical fixes

**Labels:** mechanical, medium, low

## Problem

Each fix below follows from its finding and needs no design decision, so they can be applied
together in one pass and reviewed by reading the diff. Severity still applies: a mechanical fix
 can matter a lot.

### F32 (medium): `docs/lint-codes.md` makes a `null` feedback or comment a `schema-error`

`docs/lint-codes.md:72, docs/question-types/question-base.md:103-107`

```markdown
`null` in these fields is a `schema-error`.
```

**Suggested fix:** Change the sentence to: '`null` in an OPTIONAL field is an absent field; in
a required field it is a `schema-error`.'

### F8 (low): `uniform-true-false-answers` is not described in the prose

`docs/question-types/true-false.md:250`

```markdown
| choices[].correct  | info     | uniform-true-false-answers               | the statements are
 all true or all false                    |
```

**Suggested fix:** Add a sentence, for example under Grading: 'Implementations MAY report, at
the info level, a question whose statements are all true or all false (`uniform-true-false-
answers`).'

### F9 (low): Abstention is called skip, unmarked, missing and unjudged

`docs/question-types/true-false.md:183, docs/question-types/true-false.md:201-211,
docs/question-types/true-false.md:218-225`

```markdown
Including abstentions is deliberate: a student who skips a statement because
```

**Suggested fix:** Use 'abstains on a statement' / 'abstention' throughout, and say in the
table intro that `_` is an abstention.

### F10 (low): False-friend footnote speaks of an `S` 'meaning false'

`docs/question-types/true-false.md:256-257`

```markdown
[^4]: An `S` marker meaning false under `locale: id` SHOULD get
`false-friend-true-false-marker`, see [locale](#locale).
```

**Suggested fix:** 'Every `S` marker under `locale: id` SHOULD get `false-friend-true-false-
marker`.' Attach the footnote only to that row.

### F11 (low): TRUE letters written as one code span `TVS`

`docs/question-types/true-false.md:83`

```markdown
* **TRUE**: `TVS`, plus
```

**Suggested fix:** `T`, `V`, `S`, `U+5BF9` and `U+771F`; the same for the FALSE line.

### F30 (low): Schema `marker` description omits the agreement check
````

`schema/true-false.yaml:88-93`

```yaml
carries the meaning, but the letter itself is needed to warn
          about WARNING letters and locale mismatches, and to
          round-trip the document unchanged.
```

Suggested fix: Add: 'When present, its category must agree with `correct`.'

F31 (low): Schema `grading` description ignores the exam's grading map

`schema/true-false.yaml:33-37`

```yaml
`inherit`, the default, takes the exam's `grading` value, and
          is `symmetric` outside an exam.
```

Suggested fix: 'takes the exam's `grading` (its `true-false` entry when it is a map), and
is `symmetric` outside an exam or when the exam sets none.'

F33 (low): `uniform-true-false-answers` is attached to `choices[].correct`, but reported on
 `choices`

`docs/question-types/true-false.md:250`

```markdown
| choices[].correct  | info     | uniform-true-false-answers               | the statements are
 all true or all false                    |
```

Suggested fix: Change the Field column to `choices`.

F35 (low): The introduction calls true statements 'correct answers'

`docs/question-types/true-false.md:5-7`

```markdown
rendition of a True or False marker. The correct answers are usually marked with
`[T]` and incorrect ones are usually marked with an `[F]`.
```

Suggested fix: 'True statements are usually marked with `[T]` and false statements with
`[F]`.'

F36 (low): The annex is said to hold a rationale that it does not have

`docs/question-types/true-false.md:78-81`

```markdown
[annex/true-false-spellings.md](../annex/true-false-spellings.md)
for the source data and the rationale behind each addition.
```

Suggested fix: 'for the source data. The rationale follows below.'

F37 (low): `S` is called the initial of words written in non-Latin scripts

`docs/question-types/true-false.md:87-88`

```markdown
`S` was chosen for TRUE over FALSE deliberately: it is the initial letter for
"true" in Hindi, Bengali, Arabic, Urdu and Japanese
```

Suggested fix: 'it is the initial of the romanized word for "true" in Hindi, Bengali,
Arabic, Urdu and Japanese (satya, satya, sahih, sach, shin)'.

F41 (low): Corpus comment cites multiple-choice.md for the duplicate-text rule

`examples/invalid/model-only/true-false-duplicate-choice-text.yaml:1-3`

```yaml
# are compared with whitespace normalized, so this is `duplicate-choice-text`
# (multiple-choice.md, "Choices"), which JSON Schema cannot express.
```

Suggested fix: Cite `references/choices.md, "Choice text"`.

Acceptance criteria

- [ ] F32: `docs/lint-codes.md` makes a `null` feedback or comment a `schema-error`
- [ ] F8: `uniform-true-false-answers` is not described in the prose
- [ ] F9: Abstention is called skip, unmarked, missing and unjudged
- [ ] F10: False-friend footnote speaks of an `S` 'meaning false'
- [ ] F11: TRUE letters written as one code span `TVS`
- [ ] F30: Schema `marker` description omits the agreement check
- [ ] F31: Schema `grading` description ignores the exam's grading map
- [ ] F33: `uniform-true-false-answers` is attached to `choices[].correct`, but reported on `choices`
- [ ] F35: The introduction calls true statements 'correct answers'
- [ ] F36: The annex is said to hold a rationale that it does not have
- [ ] F37: `S` is called the initial of words written in non-Latin scripts
- [ ] F41: Corpus comment cites multiple-choice.md for the duplicate-text rule

--- END ISSUE 10 ---